

FULL-STACK DEVELOPER INTERNSHIP

Take-Home Technical Assessment

Project: News Pulse — Topic-Clustered News Timeline

Time Allowed	Format	Video Required
3 days from receipt	Take-home, work solo	Yes — 2 to 3 minutes

Overview

This assessment is designed to reflect real engineering work rather than a textbook exercise. There is no single correct solution — we are evaluating how you approach problems, handle ambiguity, and communicate your decisions, not just whether every optional item is completed.

You may use any resources you would normally use on the job, including documentation, reference material, and AI tools, provided the work and understanding are your own. Where a requirement is ambiguous, make a reasonable assumption and note it in your README.

What You're Building

News Pulse: a small system that pulls live articles from a few news RSS feeds, automatically groups related articles into topic clusters, and displays those clusters as a visual timeline.

Stack	What you'll use it for
Python	RSS ingestion, content extraction, and grouping articles by topic
Node.js	Backend REST API serving clusters, articles, and timeline data
Next.js / React	Frontend timeline visualization and cluster explorer
Deployment	Deployed live for review

Part 1 — Python: RSS Ingestion & Topic Grouping

1a. Pull From Multiple News Sources

Pull articles from at least three different news RSS feeds. Use real, public feeds — for example:

- **BBC News** — <http://feeds.bbc.co.uk/news/rss.xml>
- **NPR** — <https://feeds.npr.org/1001/rss.xml>
- **Any major outlet's public RSS** (Reuters, The Guardian, Al Jazeera, etc.)

You may use any three reputable outlets with public RSS feeds — list the ones you used in your README.

This is intended to be more involved than a single clean parse. The implementation should handle:

- **Feed format inconsistencies** — different feeds use different field names/structures (`<description>` vs `<content:encoded>`), missing `pubDate`, inconsistent date formats. Try to normalize all of them into one consistent internal schema.
- **Getting the full article, not just the summary** — RSS feeds usually only give you a short summary. For at least the headline + summary, also fetch the actual article page and pull out the main body text (`newspaper3k`, `trafilatura`, and plain `BeautifulSoup` all work fine). Some pages will fail to parse — handle that gracefully instead of letting the whole run crash.
- **Avoiding duplicates** — merging the same story across different outlets is not required (it is a stretch goal below), but repeated runs should not store the same article twice.
- **Being re-runnable** — your scraper should be something you (or a scheduler) can run repeatedly, ideally only processing new articles each time rather than starting over.

1b. Group Articles Into Topics

This is the core of the assessment: take the articles you've collected and group the ones that are about the same topic or story into clusters. How sophisticated you get here is up to you — a simple approach implemented well is preferred over a complex one that does not work reliably.

Option A — Keyword / word-overlap grouping (totally fine, no NLP background needed)

Prior experience with TF-IDF, vectors, or clustering algorithms is not required — a solid solution can be built without them:

- Pull out the meaningful words from each headline/summary (lowercase everything, strip common filler words like `the`, `is`, `and` — these are usually called “stop words” and you can find ready-made lists online).
- Compare articles by how many significant words they share — e.g. if two articles share 4+ meaningful words (“election”, “senate”, “vote”, “bill”), they're probably the same topic.
- Group articles that cross some overlap threshold into the same cluster, and give that cluster a label using its most common shared words.

This is a perfectly legitimate, real approach — plenty of production systems start exactly here.

Option B — TF-IDF based grouping (if you're comfortable with it)

If you have used scikit-learn or similar before, feel free to go this route instead:

- Compute **TF-IDF vectors** over article text (headline + summary, or full body if you extracted it).
- Use **cosine similarity** or a clustering algorithm (**KMeans**, **DBSCAN**, or a similarity-threshold approach) to group related articles.
- Auto-generate a label for each cluster — e.g. the top 2–3 TF-IDF terms, or the most representative headline.

Both options are acceptable. Evaluation is based on whether the resulting clusters are coherent and whether you can clearly explain how you arrived at them — not on which option you choose.

Whichever path you take, store: a cluster ID, a cluster label, the articles that belong to it, and each article's published timestamp — this is what will power your timeline in Part 3.

Include in Your README

- Which approach you used (keyword-overlap or TF-IDF) and why
- How you picked your thresholds/parameters (e.g. how many shared words counts as “related,” or how many clusters)
- One limitation of your approach that you noticed

Part 2 — Node.js Backend API

Build an API that serves your clusters and articles to the frontend.

Required Endpoints

Endpoint	Purpose
GET /clusters	List of topic clusters — label, article count, time range (earliest → latest article)
GET /clusters/:id	Full cluster detail with all articles, sorted chronologically
GET /timeline	Clusters formatted for plotting: label, start/end time, article count, a size/intensity metric
POST /ingest/trigger	Triggers your Python pipeline (scrape + group) as a subprocess or service call; returns a job ID
GET /ingest/status/:jobId	Lets the frontend poll job status

Expectations

- A sensible shape for timeline data specifically — think about what a charting library actually needs (start/end timestamps, not just a raw list)
- Reasonable error handling, input validation, and correct status codes (400/404/500 used appropriately)
- Config via environment variables — no hardcoded DB URLs or secrets in the code
- A working connection to whichever database your Python pipeline writes to (Postgres, MongoDB, and SQLite are all fine)

Part 3 — Next.js / React Frontend: The Timeline

This section carries significant weight — the timeline should look polished and be genuinely useful, not just a list in a div.

Required

- **A timeline visualization** — clusters plotted along a time axis, where each cluster shows as a block/marker spanning from its earliest to latest article. Use a charting library ([recharts](#), [visx](#), [vis-timeline](#)), a custom-built timeline, or a calendar-style heatmap — whatever you like. It should visually communicate “this topic was active during this window,” not just be a sorted list with dates next to it.
- **A cluster detail view** — clicking a cluster on the timeline shows its articles (headline, source, published time, link to the original article).
- **A filter by source** — let the user toggle which news sources are included.
- **A “Refresh data” button** — calls `POST /ingest/trigger`, polls status, and updates the timeline once the job completes.

Stretch Goals (Optional — Not Required to Pass)

- **Auto-refresh** — the frontend polls `/timeline` periodically and the view updates live as new clusters/articles appear, with no manual refresh needed.
- **Visual cluster sizing** — bigger cluster = bigger/bolder marker on the timeline.
- **Cross-source story merging** — recognizing the same real-world story across two outlets as one logical event, even if they remain in separate clusters. This is a known hard problem; it is not required.

Part 4 — Deployment

Deploy the full system so the deliverable can be reviewed live, not run locally.

Component	Suggested platform(s)
Frontend	Vercel or Netlify (both have generous free tiers)
Backend API	Render, Railway, or Fly.io
Python pipeline	GitHub Actions cron, a scheduled job on Render/Railway, or triggered on-demand via the Node API
Database	Supabase, Neon, MongoDB Atlas, or Railway Postgres (any hosted free tier works)

Requirements

- A live URL that works when opened cold (standard free-tier cold-start delays are acceptable)
- Environment variables configured on the hosting platform, not committed to the repository
- A brief README note on what runs where, and why

Part 5 — Video Walkthrough (Required)

Record a **2–3 minute** screen recording walking through your project. This is a required part of the submission and is evaluated alongside the code.

Order	What to cover	Suggested length
1	A live demo of your timeline — show it actually working with real current news	30–45 sec
2	How your topic grouping works — briefly show the code and explain, in plain language, how articles end up grouped together	45–60 sec
3	One hard problem you ran into (e.g. messy RSS formats, deciding what counts as “related,” or making the timeline readable) and how you solved it	30–45 sec
4	One thing you'd improve with more time	15–20 sec

Loom, OBS, or a phone screen recorder are all acceptable. Share an unlisted link with your submission.

Submission Checklist

Please ensure your submission includes the following:

- ☐ A GitHub repo link, with clearly separated /scraper, /backend, and /frontend folders
 - ☐ A live frontend URL
 - ☐ A live backend API URL (or clearly documented endpoints if not directly browsable)
 - ☐ A README covering: setup instructions, a brief architecture overview, which topic-grouping approach you used and its limitations, and which news sources you used
 - ☐ Your 2–3 minute video walkthrough link
-

A Few Notes

- There is no single correct solution — evaluation is based on approach and reasoning, not just final polish.
- If something is ambiguous, make a reasonable assumption, note it in your README, and keep moving.
- For any questions about this assessment, please reach out to us directly rather than guessing.